

파이썬 프로그래밍 5주차 요약본

21. 딥러닝 파이썬 패키지(3)

21.1 seaborn 패키지

- python의 데이터 시각화 라이브러리
- Matplotlib 위에서 동작
- 간결한 문법과 고급 시각화 기능 제공

```
# 설치
pip install seaborn
```

- 산점도 (Scatter Plot)
 - "꽃잎 길이"와 "꽃잎 너비"간의 관계를 시각적으로 표현

```
import seaborn as sns
import matplotlib.pyplot as plt

# 샘플 데이터셋 로드
iris = sns.load_dataset("iris")

# 그래프 표시
sns.scatterplot(data=iris,
                x="sepal_length",
                y="sepal_width",
                hue="species",
                style="species")

plt.title("Scatter Plot Example")
plt.show()
```

- 선형 회귀선 (lmlot, linear model)
"꽃잎 길이"와 "꽃잎 너비"의 선형적 관계를 시각적으로 표현

```
import seaborn as sns
import matplotlib.pyplot as plt

# 샘플 데이터셋 로드
iris = sns.load_dataset("iris")

# 선형 회귀 그래프 표시
sns.lmplot(data=iris,
            x="sepal_length",
            y="sepal_width",
            hue="species",
            height=6)

plt.title("Linear Regression Plot")
plt.show()
```

21.2 OpenCV

- 실시간 컴퓨터 비전을 위한 라이브러리
- Python을 포함한 여러 프로그래밍 언어에서 사용 가능
- 다양한 이미지 및 영상 처리 기능 제공

```
# 설치
pip install opencv-python
```

- 기본 이미지 로드 및 표시

```
import cv2

# 이미지 로드
image = cv2.imread('./ch21/opencv/sample.jpg')

# 이미지를 OpenCV 창에서 표시
cv2.imshow('Loaded Image', image)
# 키 입력을 기다린 후 창 닫기
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- 얼굴 검출

```
import cv2
import matplotlib.pyplot as plt

# 이미지 로드
image = cv2.imread('./ch21/opencv/people.jpg')

# 얼굴 검출을 위한 Haar Cascade 파일 로드
face_path = cv2.data.harcascades +
            'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(face_path)

# 이미지를 그레이스케일로 변환
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 얼굴 검출 수행
faces = face_cascade.detectMultiScale(gray,
                                       scaleFactor=1.1,
                                       minNeighbors=5,
                                       minSize=(30, 30))

# 검출된 얼굴 주위에 사각형 그리기
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y),
                  (x + w, y + h), (255, 0, 0), 2)

# 결과 출력
cv2.imshow('Face Detection', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- 실시간 비디오(웹캠) 영상 처리

```

import cv2

# 웹캠 사용 (필요시 주석 해제)
# cap = cv2.VideoCapture(0)

# 동영상 파일 사용
cap = cv2.VideoCapture("./ch21/opencv/sample.mp4")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Canny 엣지 검출 적용
    edges = cv2.Canny(frame, 100, 200)

    # 결과 출력
    cv2.imshow("Edge Detection", edges)

    # 'q' 키를 누르면 종료
    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

22. Python 활용 심화 (1)

22.1 파일관리 기본(os, shutil 활용)

- OS 모듈
 - os 모듈은 운영 체제와 상호 작용하는 기능을 제공
 - 주요 기능

```

# 라이브러리 импорт
import os
# 현재 작업 디렉토리 확인
print("현재 작업 디렉토리:", os.getcwd())
# 작업 디렉토리 변경
os.chdir("./ch22/os_module")
# 현재 디렉토리의 파일 및 폴더 목록 출력
print("디렉토리 목록:", os.listdir("."))
# 디렉토리 생성
os.mkdir("test_dir")
# 디렉토리 삭제
os.rmdir("test_dir")
# 파일 존재 여부 확인
if os.path.exists("file.txt"):
    print("파일이 존재합니다.")

```

- **shutil 모듈**

- 파일 및 디렉토리 관리
- 주요 기능

```

# 라이브러리 импорт
import shutil
shutil.copy("source.txt", "destination.txt") # 파일 복사
shutil.copy2("source.txt", "destination.txt") # 메타데이터 유지하며 복사
# 디렉토리 전체 복사
shutil.copytree("source_dir", "destination_dir")
# 파일 및 디렉토리 이동
shutil.move("old_location", "new_location")
# 파일 및 디렉토리 삭제
# 디렉토리 및 하위 파일 모두 삭제
shutil.rmtree("directory_to_delete")

```

22.2 파일 정리 프로그램 제작(pathlib 활용)

- **pathlib 모듈**

- 객체 지향 방식으로 파일 및 디렉터리 관리 가능
- **주요 기능**

```
# 라이브러리 импорт
from pathlib import Path
# 현재 작업 디렉토리 확인
print(Path.cwd())
# 경로 생성 및 조작
path = Path("folder")
# 폴더와 파일 경로 결합
# '/' 연산자는 경로 결합 연산자(pathlib.Path 객체에서 지원)
print(path / "file.txt")
# 디렉토리 및 파일 존재 여부 확인
path = Path("file.txt")
path = Path("folder")
print(path.exists()) # 존재 여부 확인
print(path.is_file()) # 존재 및 파일 여부 확인
print(path.is_dir()) # 디렉토리 여부 확인
# 디렉토리 생성 및 삭제
path = Path("new_folder")
path.mkdir(exist_ok=True) # 폴더 생성(기존 존재 시, 에러발생 없음)
path.rmdir() # 폴더 삭제
# 파일 생성 및 삭제
file_path = Path("test.txt")
file_path.touch() # 빈 파일 생성
file_path.unlink() # 파일 삭제
# 파일 및 폴더 목록 조회
path = Path(dir)
for item in path.iterdir():
    print(item)
# 파일 확장자 가져오기
path = Path(file)
print(path.suffix) # 출력: .txt
# 파일 이름 변경 및 이동하기
path = Path("example.txt")
path.touch() # 파일 생성
destination = Path("new_folder/example.txt")
```

```
estimation.parent.mkdir(exist_ok=True) # 대상 폴더 생성
path.rename(destination)
```

22.3 Excel 읽기/쓰기 기초(openpyxl)

- 파이썬에서 Excel 파일을 읽고 쓰기 위해 사용되는 라이브러리
- 주요 기능

```
# 라이브러리 импорт
import openpyxl
from openpyxl import Workbook
from openpyxl import load_workbook
# 새 Excel 파일 생성
Workbook(): 새 Excel 파일을 생성
sheet['A1'] = '이름': A1 셀에 "이름"을 입력
wb.save('new_example.xlsx'): 새 파일을 저장
wb = Workbook() # 새로운 워크북 생성
sheet = wb.active # 활성 시트 선택
# 셀에 데이터 입력
sheet['A1'] = 'Hello'
sheet['B1'] = 'World'
# 저장하기
wb.save('new_example.xlsx')
```

22.4 Excel 데이터 정리 자동화(openpyxl, pandas 활용)

```
# 라이브러리 импорт
import pandas as pd
import openpyxl
# pd.read_excel("파일명.xlsx", sheet_name="Sheet1")
df.head()
```

22.5 웹 데이터 가져오기(requests, BeautifulSoup)

- 웹에서 데이터를 자동으로 가져와서 활용하는 기술
- 사용 모듈
 - requests: 웹페이지 요청을 보내고 HTML을 가져옴

- BeautifulSoup4: 가져온 HTML을 분석하고 필요한 데이터를 추출

```
# 라이브러리 импорт
import requests
from bs4 import BeautifulSoup
url = "https://example.com" # 크롤링할 웹페이지 URL
response = requests.get(url) # GET 요청으로 페이지 가져오기
# 응답 코드 출력 (200 정상)
print(response.status_code)
# 가져온 HTML 일부 출력 (첫 500자)
print(response.text[:500])
```

22.6 간단한 데이터 수집 및 저장 프로그램

```
import pandas as pd
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
}
# DataFrame 생성
df = pd.DataFrame(data)
# CSV 파일로 저장
file = "./ch22/save_file/output.csv"
df.to_csv(file, index=False) # 인덱스 열 저장하지 않음
print(f"Data has been written to {file}")
```

23. Python 활용 심화 (2)

23.1 JSON과 API 활용 기초

- JSON(JavaScript Object Notation)
 - JSON은 데이터를 저장하고 전송하는 데 사용하는 경량 데이터 형식
 - JSON 예시 (sample.json)


```
{
  "name": "lice",
  "age": 25,
  "city": "Seoul",
  "skills": ["Python", "Django", "Machine Learning"]
}
```

- **JSON 문자열을 파이썬 객체로 변환 예**

```
import json
json_string = '{"name": "Alice", "age": 25, "city": "Seoul"}'
• # JSON 문자열 -> 딕셔너리 변환
• data = json.loads(json_string)
• print(data["name"]) # Alice 출력
```

- **파이썬 객체를 JSON 문자열로 변환**

```
python_dict = {"name": "Bob", "age": 30, "city": "Busan"}

# JSON 포맷 변환
json_output = json.dumps(python_dict, indent=4)
print(json_output)
```

- **JSON 파일 입출력**

```
# JSON 파일 저장
with open("data.json", "w") as file:
    json.dump(python_dict, file, indent=4)

# JSON 파일 불러오기
with open("data.json", "r") as file:
    loaded_data = json.
```

- **API(Application Programming Interface)**

- 프로그램 간 데이터와 기능을 주고받을 수 있도록 하는 인터페이스
- REST API: HTTP 프로토콜을 기반으로 JSON 형태의 데이터를 주고받는 방식

23.2 API 데이터 활용

- 날씨 데이터 그래프로 출력하기
 - 여러 도시들의 최저 온도를 산점도로 표시

```
import matplotlib.pyplot as plt

plt.rcParams['font.family'] = 'Malgun Gothic' # 한글 표시
plt.rcParams['axes.unicode_minus'] = False # 음수('-') 표시
cities = ["Seoul", "New York", "Tokyo", "Sydney"]
k2c = lambda k: k - 273.15

# 데이터를 저장할 리스트
temp_min_list = []
temp_max_list = []
temp_min_list.append(k2c(data["main"]["temp_min"]))
temp_max_list.append(k2c(data["main"]["temp_max"]))

# 최저 기온 산점도
plt.scatter(cities, temp_min_list, color='blue',
            label='최저 기온', s=100, marker='o')
plt.title('도시별 최저 기온')
plt.xlabel('도시')
plt.ylabel('기온 (°C)')
plt.ylim(-15, 30) # y축 범위 설정
plt.xlim(-0.5, len(cities) - 0.5) # x축 여백 추가
plt.grid(True)
plt.legend()

# 그래프 표시
plt.tight_layout()
plt.show()
```

23.3 정규 표현식 활용1

- 정규 표현식
 - 문자열에서 특정 패턴을 찾거나, 검사하거나, 치환하기 위한 표현식
 - 기본 문법 예시

패턴	의미	예시 ("abc123")
.	임의의 한 글자	a.c → "abc" 일치
*	0회 이상 반복	ab* → "a", "ab", "abb"
+	1회 이상 반복	ab+ → "ab", "abb"
?	0 또는 1회	colou?r → "color", "colour"
[]	문자 집합	[0-9] → 숫자 하나
^	문자열 시작	^abc → "abc123" 일치
\$	문자열 끝	123\$ → "abc123" 일치
		OR 조건
()	그룹	(abc)+ → "abcbac" 가능
\d	숫자(= [0-9])	\d+ → "123"
\w	문자/숫자/밑줄	\w+ → "abc123"
\s	공백 문자	\s+ → " "

◦ 유효 이메일 검증

```
import re

email = "test@example.com"
pattern = r"^[w.-]+@[w.-]+\.[a-zA-Z]{2,}$"
if re.match(pattern, email):
    print("유효한 이메일 주소입니다.")
else:
    print("잘못된 이메일 주소입니다.")

# 실행 결과
# 유효한 이메일 주소입니다.
```

23.4 데이터 시각화

• 데이터 시각화(Data Visualization)

- 숫자나 텍스트 형태의 데이터를 사람이 이해하기 쉽게 그래프, 차트, 지도 등으로 표현하는 과정

- 복잡한 데이터를 그림으로 보여줘서 **패턴, 추세, 이상치** 등을 쉽게 발견할 수 있도록 도와줌
- **대표적인 시각화 종류**
 - 막대 그래프, 선 그래프, 파이 차트, 히트맵, 산점도
- **Python의 데이터 시각화 모듈**
 - **Matplotlib** → 가장 기본적인 시각화 라이브러리
 - **Seaborn** → 통계적 시각화에 강함
 - **Plotly** → 인터랙티브 그래프 가능
 - **Pandas** → DataFrame 바로 시각화 가능
- **파이썬 데이터 시각화 예제**

```
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4, 5]
y1 = [10, 20, 25, 30, 40]
y2 = [8, 18, 28, 22, 35]

# 막대 그래프 생성
plt.bar(x, y1)
plt.title("막대 그래프 예제")
plt.xlabel("X 값")
plt.ylabel("Y 값")
plt.show()
```

23.5 데이터 통합 및 보고서 자동화

- **데이터 통합 (Data Integration)**
 - 여러 출처의 데이터를 모아 하나의 일관된 형태로 정리하는 과정
 - 보통 데이터가 **엑셀, 데이터베이스, API, 로그 파일** 등 다양한 형태로 흩어져 있기 때문에, 이를 합쳐야 분석이 가능
- **보고서 자동화 (Report Automation)**
 - 데이터 분석 및 시각화를 자동으로 수행하고, 결과를 정기적으로 보고서 형태로 생성하는 과정

- 사람이 매번 엑셀 편집하거나 PPT 만드는 대신, 코드나 도구가 자동으로 처리